Homework 2: ER Modeling

COSC 3337 · Fall 2026

Due: Sunday, October 18, 2026, 11:59 PM · **Points:** 100 (+5 bonus)

Overview

In this homework you will design a database for a small business, starting from what its application needs to do. You will draw an ER diagram, reduce it to a relational schema, and then build that schema in MySQL and test whether it actually enforces the business's rules.

By the end you should be able to:

Submission

Submit a single file, `hw2.zip`, containing **exactly two files at the top level**:

- `hw2.pdf`: all of Part 1.
- `schema.sql`: your schema for Part 2.

Zip the two files themselves, not a folder containing them (`zip hw2.zip hw2.pdf schema.sql`). Handwritten submissions must be scanned properly; do not submit a photograph of your work. **LaTeX bonus (+5):** typeset `hw2.pdf` in LaTeX. Diagrams may be drawn by hand or with a tool such as draw.io and included as images.

Before you start

Read the **naming contract** in Part 2 (page 6) before you draw your diagram in Part 1. Your ER diagram and schema should use those names, because your Part 2 code is built from your Part 1 design.

Part 0: Set Up Python on Your VM (not graded)

The self-check script for Part 2 is written in Python and needs the package `mysql-connector-python`. In this part you install **Miniconda**, a small Python distribution, in your home directory on your VM, and create a Python environment for this course. Run every command below in an SSH session on your VM. It takes about five minutes.

Step 1: Download and install Miniconda

```
mkdir -p ~/miniconda3
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh \
     -O ~/miniconda3/miniconda.sh
bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3
rm ~/miniconda3/miniconda.sh
```

The `-b` flag installs without asking questions, and `-p` sets the install location. Nothing here needs `sudo`.

Step 2: Turn on conda

```
source ~/miniconda3/bin/activate
conda init bash
```

Now **log out and SSH back in**. Your prompt should start with `(base)`, which means conda is active.

Step 3: Create an environment for this course

```
conda create -n cosc3337 python=3.12 -y
conda activate cosc3337
```

Your prompt should now start with `(cosc3337)`.

Step 4: Install the required package

```
pip install mysql-connector-python
```

Step 5: Check that everything works

```
which python
python -c "import mysql.connector; print(mysql.connector.__version__)"
mysql --version
```

`which python` should print a path inside `~/miniconda3/envs/cosc3337/`. The second command should print a version number with no errors. The script also uses the `mysql` command-line client, which is already installed on your VM along with MySQL.

Every time you log in to work on this homework, run `conda activate cosc3337` first.

Troubleshooting

- `wget: command not found`: run `sudo apt install wget`, then repeat Step 2.
- `conda: command not found` **after logging back in**: run `source ~/miniconda3/bin/activate` and `conda init bash` again, then log out and back in.
- **An error about Terms of Service during Step 3**: run the two commands below, then repeat Step 3.

```
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
```

- `ModuleNotFoundError: No module named 'mysql'`: your course environment isn't active. Run `conda activate cosc3337` and check that `which python` points into `~/miniconda3/envs/cosc3337/`.
- **Your VM was replaced**: a new VM doesn't have Miniconda, so repeat Part 0.
- **OOM errors installing miniconda**

```
fallocate -l 2G /swapfile    # create an empty 2 GB file
chmod 600 /swapfile          # restrict access to root (required for swap files)
mkswap /swapfile             # format the file as swap space
swapon /swapfile             # turn the swap on
free -h                      # verify: the Swap row should show ~2.0Gi
echo '/swapfile none swap sw 0 0' >> /etc/fstab   # turn swap on after a reboot
```

The Scenario: Blue Heron Dive Co.

The owner of a (fictional) local dive shop is hiring a developer to build a booking app, and needs a database first. She described what the app has to do, screen by screen. You will use this brief in Part 1 and Part 2.

Hi! I'm Marisol, and I own Blue Heron Dive Co. We run boat dive trips on Lake Travis and at a few spring-fed sites in the Hill Country. Right now everything lives in one giant spreadsheet, and it's falling apart. Here's what the new app needs to do.

Screen 1: Customer profile. Shows the customer's name; their email (they log in with it, so no two customers can share one); their mailing address (street, city, state, ZIP); every phone number they've given us (some people give us one, some give us three); their certification level, which is one of *Open Water*, *Advanced*, or *Rescue*; and their upcoming trips.

Screen 2: Trip page. Shows the trip's date and price, the boat it goes out on, and the divemaster leading it. Every trip uses exactly one boat and is led by exactly one divemaster. The page lists every customer booked on the trip and how much each has paid so far: deposits are fine, but nobody can pay a negative amount. It also shows *spots remaining*, which is the boat's capacity minus the number of bookings. A customer can't book the same trip twice.

Screen 3: Boats and staff. Each boat has a name and a capacity, which is at least one seat. We never remove a boat that has trips on record, because we need that history for insurance. Each divemaster has a name and a PADI professional number, which is unique to that person.

Screen 4: Dive log. Each trip has one or more dives, numbered 1, 2, 3, ... within that trip. "Dive 2" only means something together with its trip. For each dive we log the maximum depth in meters (never deeper than 40 m, the recreational limit) and the bottom time in minutes. Each dive happens at a dive site, which has a name and an area. A trip usually stays at one site, but sometimes the boat moves between dives. If we cancel a trip, its dive log goes with it.

Screen 5: Gear desk. We have rental gear items, each with a serial number, a type (BCD, regulator, wetsuit, fins, or dive computer), and a size where that applies. Staff record which customer rented which item for which trip, and the rental fee. An item can only go to one customer on a given trip.

Glossary for non-divers

Trip: one boat outing on a given date.
Dive: one descent underwater. A trip usually includes two or three dives.
Divemaster: a dive professional employed by the shop to lead trips.
PADI professional number: an ID number issued to each dive professional by a certification agency (PADI).
Certification level: the level of training a diver has completed.
Bottom time: how long a dive lasts, in minutes.
BCD, regulator, dive computer: types of scuba equipment.

Part 1: Written (50 pts)

Use the ER notation from class.

Question 1.1: Model from Requirements (30 pts)

Draw a complete ER diagram for Blue Heron Dive Co. that supports all five screens.

(a) **ER diagram.** Include every entity set, relationship set, and attribute the screens need, with keys, cardinality constraints, and participation constraints. Use composite, multivalued, and derived attributes where they fit.

(b) **Assumptions log.** The brief is ambiguous or incomplete in at least three places. List every assumption you made, each with a one-sentence justification that refers to the brief. For example: *"I assume a dive site belongs to a dive rather than a trip, because the boat sometimes moves between dives (Screen 4)."* (You may use that one, but it doesn't count toward your three.)

Rubric item	Points
Entity sets and attributes (including composite, multivalued, derived)	7
Relationship sets, including any relationship of degree > 2	6
Cardinality and participation constraints	6
Keys, including the weak entity set and its discriminator	5
Assumptions log: ambiguities found and resolved with justification	6

Question 1.2: Reduce Your ER Diagram to a Relational Schema (20 pts)

Reduce **your own** diagram from Question 1.1 to a relational schema using the rules in Silberschatz §6.7.

- Write one line per relation, with primary-key attributes underlined and each foreign key marked with the relation it references. For example:

  `trip(`<u>`trip_id`</u>`, boat_id → boat, divemaster_id → divemaster, trip_date, price)`

- Next to each relation, note in a few words which rule produced it (e.g., "strong entity set," "many-to-many relationship set," "multivalued attribute").
- If you combined a relationship set into another relation instead of giving it its own (e.g., a many-to-one relationship with total participation), say so.
- Use the table and column names from the naming contract (page 6).

Rubric item	Points
Correct set of relations for your diagram	8
Primary keys	4
Foreign keys	4
Special cases: weak entity, multivalued/composite attributes, degree > 2	4

Part 2: Coding (50 pts)

In this part you implement your Question 1.2 schema in MySQL on your VM and test whether it enforces Blue Heron's rules.

How Part 2 is graded

The autograder only checks functionality: whether `schema.sql` loads, whether the required tables, keys, and constraints exist, and whether your schema accepts and rejects the test inserts listed below. All of its checks are listed in this handout, and you can run them yourself with the self-check script.

I will also read your `schema.sql` and grade it for design. Passing every autograder check does not guarantee full credit. For example, a schema that passes the tests but doesn't match your Part 1 diagram, uses poor data types, or enforces a rule in a way that doesn't make sense will lose design points.

The Naming Contract

The autograder needs to know what your tables are called. Your schema must contain the following tables with **at least** these columns, using exactly these names:

Table	Required columns
`customer`	`customer_id`, `email`, `first_name`, `last_name`, `cert_level`
`divemaster`	`divemaster_id`, `first_name`, `last_name`, `pro_number`
`boat`	`boat_id`, `name`, `capacity`
`trip`	`trip_id`, `boat_id`, `divemaster_id`, `trip_date`, `price`
`dive`	`trip_id`, `dive_no`, `max_depth_m`, `bottom_time_min`
`booking`	`customer_id`, `trip_id`, `amount_paid`

The contract fixes *names only*. It does not say which columns are keys, what data types to use, or what constraints to add. Those are your design decisions.

- You may (and should) add other tables, such as those for phone numbers, dive sites, gear, and rentals, and add other columns to the tables above.
- **Any column you add to a contract table must allow `NULL` or have a `DEFAULT`.** The tests insert rows using only the contract columns, so an extra `NOT NULL` column without a default would make every insert fail.
- IDs are inserted explicitly by the tests (e.g., `customer_id = 1`), so don't rely on `AUTO_INCREMENT` values.

Rules for `schema.sql`

- It must begin with exactly these three lines:

```
DROP DATABASE IF EXISTS hw2_dive;
CREATE DATABASE hw2_dive;
USE hw2_dive;
```

- It contains only DDL: `CREATE TABLE` statements (and `ALTER TABLE` if you need it). **No `INSERT` statements**; the tests load their own data.
- Comment your code: a short comment above each table saying which part of your ER diagram it comes from is enough.

Question 2.1: DDL from Your Design (15 pts, autograded)

Write `schema.sql` implementing your Question 1.2 schema, with primary keys, foreign keys (including any `ON DELETE` behavior the brief calls for), `NOT NULL`, `UNIQUE`, and `CHECK` constraints wherever the brief's rules require them.

The autograder loads your file into a fresh database and runs these checks:

	Check	Pts
S1	`schema.sql` runs without errors. *If this fails, nothing else can be checked and Part 2's autograded score is 0.*	3
S2	All contract tables and columns exist; every extra column in a contract table allows `NULL` or has a `DEFAULT`	3
S3	Every table has a primary key	2
S4	The primary key of `dive` is exactly (`trip_id`, `dive_no`)	2
S5	Foreign keys exist from `trip` → `boat`, `trip` → `divemaster`, `booking` → `customer`, `booking` → `trip`, and `dive` → `trip`	3
S6	At least 3 `CHECK` and at least 2 `UNIQUE` constraints in total	1
S7	Every table uses the InnoDB storage engine (the MySQL 8 default; foreign keys are not enforced otherwise)	1

Question 2.2: Constraint Test Suite (20 pts, autograded)

Each test below is a business rule from the brief turned into one SQL statement. Your schema should **accept** the valid statements and **reject** the invalid ones **with the right MySQL error**. A statement that fails for some other reason (a syntax error, an unknown column, a different constraint) does not pass.

Before the tests run, the autograder loads this data into your schema:

```
customer:   (1, 'ana@example.com', 'Ana', 'Diaz', 'Advanced'),
            (2, 'ben@example.com', 'Ben', 'Okafor', 'Open Water')
divemaster: (1, 'Carla', 'Ruiz', 'PRO-1001')
boat:       (1, 'Manta', 12)
trip:       (1, boat 1, divemaster 1, '2026-11-14', 95.00)
dive:       (trip 1, dive_no 1, max_depth_m 18, bottom_time_min 45)
booking:    (customer 1, trip 1, amount_paid 95.00)
```

Each test runs inside its own transaction and is rolled back afterwards, so tests don't affect each other. Each test is worth 1 point. The exact SQL for every test is in the self-check script.

	Test	**Expected result**
T01	Add a new customer with certification level *Rescue*	accepted
T02	Add a second trip on the same boat with the same divemaster	accepted
T03	Customer 2 books trip 1 with a partial deposit	accepted
T04	Add dive 2 to trip 1	accepted
T05	Dive number 1 is used again, on a different trip	accepted
T06	Customer 1, who already booked trip 1, books a second trip	accepted
T07	Delete a trip that has dives but no bookings; its dives are deleted too	accepted; 0 dives remain
T08	Two customers with the same email	rejected: 1062 (duplicate)
T09	Certification level *Expert*	rejected: 3819 (CHECK) or 1265 (ENUM)
T10	Boat with capacity 0	rejected: 3819 (CHECK)
T11	Two divemasters with the same PADI pro number	rejected: 1062 (duplicate)
T12	Trip with no boat (`boat_id` is `NULL`)	rejected: 1048 (NOT NULL)
T13	Trip on a boat that doesn't exist	rejected: 1452 (foreign key)
T14	Booking for a customer who doesn't exist	rejected: 1452 (foreign key)
T15	Booking for a trip that doesn't exist	rejected: 1452 (foreign key)
T16	The same customer books the same trip twice	rejected: 1062 (duplicate)
T17	Booking with a negative amount paid	rejected: 3819 (CHECK) or 1264 (out of range)
T18	Two dives with the same number on the same trip	rejected: 1062 (duplicate)
T19	A dive with a maximum depth of 45 m	rejected: 3819 (CHECK)
T20	Delete boat 1, which has a trip on record	rejected: 1451 (foreign key)

Schema Design (15 pts, graded by hand)

I will read your `schema.sql` and grade it on:

- **Consistency with Part 1**: the tables and keys match your Question 1.2 schema and your assumptions log. If you changed your design while coding, update Part 1 to match.
- **Appropriate data types**: e.g., `DATE` for dates, `DECIMAL` rather than `FLOAT` for money, sensible `VARCHAR` lengths.
- **Constraints that express the rules**: every rule in the brief that *can* be enforced with a key, `NOT NULL`, `UNIQUE`, `CHECK`, or `ON DELETE` behavior is enforced, including the tables the tests don't touch (phones, sites, gear, rentals).

- **Readability**: named constraints (e.g., `CONSTRAINT ck_boat_capacity CHECK (...)`), consistent formatting, and comments linking each table to your diagram.

Testing on Your VM

The self-check script `hw2_check.py`, posted with this assignment, runs **exactly the same checks as the autograder** and prints your autograded score. Run it before you submit.

Complete Part 0 first. Then, from the folder that contains `schema.sql` and `hw2_check.py`:

```
conda activate cosc3337
python hw2_check.py schema.sql
```

Run this as the same Linux user you use to open MySQL with the `mysql` command (on most course VMs, `root`); the script then logs in to MySQL the same way, with no password. If you log in to MySQL with a username and password instead, add them:

```
python hw2_check.py schema.sql --user YOUR_MYSQL_USER --password YOUR_MYSQL_PASSWORD
```

If the script says it can't connect to MySQL, start the server with `sudo systemctl start mysql` and run it again.

The script drops and recreates the database `hw2_dive` every time it runs.

Keep a copy off your VM

Your VM can't be restored if something goes wrong; the only fix is a fresh VM. Keep `schema.sql` (and everything else for this homework) on your own computer or in a Git repository, and copy it to the VM to test.

Grading Summary

Part	Question	Points
1: Written	1.1 Model from requirements	30
	1.2 Reduce to a relational schema	20
2: Coding	2.1 DDL structural checks (autograded)	15
	2.2 Constraint test suite (autograded)	20
	Schema design (graded by hand)	15
	Total	**100**
	LaTeX bonus	+5